Para diseñar la arquitectura de base de datos de un SaaS B2B en **PostgreSQL**, la mejor opción en cuanto a balance de seguridad, aislamiento de datos y costos fijos bajos es el enfoque de Multi-Tenancy por Esquema (Schema-based Multi-Tenancy).

En lugar de crear una base de datos física distinta para cada cliente (lo que dispararía tus costos en la nube) o mezclar todos los datos en las mismas tablas con un `tenant_id` (lo que arriesga que por un error de código un cliente vea datos de otro), crearemos **un esquema de PostgreSQL aislado por cada cliente dentro de la misma base de datos**.

A continuación, se detalla la arquitectura de la base de datos central y el flujo de aislamiento:

🗺️ 1. Estructura de la Base de Datos Central

Tendrás un esquema principal llamado `public` que guardará la información de control del SaaS (los clientes corporativos, sus suscripciones y credenciales de conexión). Cada nuevo cliente que se registre disparará la creación de su propio esquema (ej. `tenant_empresa_a`).

Tabla de Control Global (Esquema `public`)

Estas tablas residen en el esquema por defecto y sirven para que tu backend en **Spring Boot** identifique a qué esquema o base de datos externa debe redirigir la consulta de IA.

sql

```
-- Crear tabla de empresas clientes (Tenants)
CREATE TABLE public.tenants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre_empresa VARCHAR(150) NOT NULL,
    subdominio_slug VARCHAR(50) UNIQUE NOT NULL, -- Ej: 'empresa_a'
para usar esquema tenant_empresa_a
    fecha_registro TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) DEFAULT 'ACTIVE' -- ACTIVE, SUSPENDED, CANCELED
);

-- Registrar las suscripciones de los clientes
CREATE TABLE public.suscripciones (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES public.tenants(id) ON DELETE CASCADE,
    plan_tipo VARCHAR(30) NOT NULL, -- STARTER, GROWTH, ENTERPRISE
    api_tokens_usados INT DEFAULT 0,
    limite_consultas INT NOT NULL,
```

```
    proxima_facturacion DATE NOT NULL
);

-- Configuración de conexión del cliente (Cifrada si guardas
credenciales externas)
CREATE TABLE public.tenant_db_connections (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES public.tenants(id) ON DELETE CASCADE,
    host VARCHAR(255) NOT NULL,
    puerto INT DEFAULT 5432,
    db_name VARCHAR(100) NOT NULL,
    db_user VARCHAR(100) NOT NULL,
    db_password_encrypted TEXT NOT NULL, -- Contraseña cifrada en
reposo
    tipo_conexion VARCHAR(20) DEFAULT 'INTERNAL_SCHEMA' --
INTERNAL_SCHEMA o EXTERNAL_DB (Enterprise)
);
```

Usa el código con precaución.

🔒 2. Estructura de un Esquema Aislado (El "Tenant")

Cuando una empresa se da de alta, tu backend ejecuta dinámicamente un script para crear su base de datos aislada. Esto evita por completo la fuga de datos:

sql

```
-- Paso 1: Crear el esquema exclusivo para el cliente 'empresa_a'
CREATE SCHEMA tenant_empresa_a;

-- Paso 2: Crear las tablas de negocio DENTRO de ese esquema exclusivo
CREATE TABLE tenant_empresa_a.usuarios (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    rol VARCHAR(30) -- ADMIN, VIEWER
);

CREATE TABLE tenant_empresa_a.consultas_historial (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id INT,
    pregunta_usuario TEXT NOT NULL,
    query_sql_generada TEXT NOT NULL,
    fecha_consulta TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

Usa el código con precaución.

⚙️ 3. Implementación del Tenant Resolver en Spring Boot

Para que Spring Boot sepa a qué esquema de PostgreSQL debe dirigir las consultas SQL sin tener que reiniciar el servidor ni hardcodear rutas, debes implementar un enrutador dinámico utilizando las interfaces nativas de Hibernate (`CurrentTenantIdentifierResolver` y `MultiTenantConnectionProvider`).

Paso A: Interceptor para capturar el Tenant del Request

Cada petición HTTP de tus clientes debe llevar un encabezado (ej. `X-Tenant-ID`) o resolverse mediante el subdominio URL (`empresa_://tusass.com`).

java

```
public class TenantInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest request,
HttpServletResponse response, Object handler) {
        String tenantId = request.getHeader("X-Tenant-ID"); // O
extraer del subdominio
        if (tenantId != null) {
            TenantContext.setCurrentTenant(tenantId);
        } else {
            TenantContext.setCurrentTenant("public");
        }
        return true;
    }

    @Override
    public void postHandle(HttpServletRequest request,
HttpServletResponse response, Object handler, ModelAndView
modelAndView) {
        TenantContext.clear();
    }
}
```

Usa el código con precaución.

Paso B: Configurar Hibernate para cambiar el Esquema dinámicamente

Le indicamos a PostgreSQL que ejecute el comando `SET search_path TO tenant_id` cada vez que una conexión de base de datos se activa para resolver un request.

java

```
public class SchemaMultiTenantConnectionProvider extends
AbstractMultiTenantConnectionProvider {
    private final DataSource dataSource;

    public SchemaMultiTenantConnectionProvider(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    protected DataSource selectAnyDataSource() {
        return dataSource;
    }

    @Override
    protected DataSource selectDataSource(Object tenantIdentifier) {
        return dataSource;
    }

    @Override
    public Connection getConnection(Object tenantIdentifier) throws
SQLException {
        final Connection connection =
selectAnyDataSource().getConnection();
        // Forzar el cambio de esquema en PostgreSQL antes de ejecutar
cualquier consulta de este hilo
        try (Statement statement = connection.createStatement()) {
            statement.execute("SET search_path TO " +
tenantIdentifier);
        } catch (SQLException e) {
            throw new HibernateException("No se pudo cambiar al
esquema: " + tenantIdentifier, e);
        }
        return connection;
    }
}
```

Usa el código con precaución.

🚀 Ventajas de esta Arquitectura para tu SaaS de Datos e IA

1. **Seguridad Absoluta:** Si la IA genera una query debido a un prompt del usuario, la query solo buscará tablas dentro de su `search_path` (esquema del cliente). Es matemáticamente imposible que acceda a los datos de la Empresa B.
2. **Migraciones con Liquibase/Flyway:** Puedes automatizar tus scripts de actualización para que se ejecuten en bucle sobre todos los esquemas existentes de forma ordenada.
3. **Fácil Respaldo:** Si la Empresa A te pide una copia de sus datos, simplemente exportas su esquema individual usando `pg_dump -n tenant_empresa_a` en un solo comando rápido, sin tocar el resto del SaaS.